Linux网络----Tcp协议(传输控制协议)

理解封装和解包：

> 本质上就是Linux内核中的指针移动操作！！！！
>
> struct sk_buffer{
>
> struct sk_buffer* next;
>
> char* **head**;
>
> char* data;
>
> ...
>
> }
>
> 假设udp报头结构体为struct udp_head{.....};
>
> 简单演示提取过程：(udp_head*)**head**->srcport = xxx;(udp_head*)head->destport =yyyy;.....

先提出2个问题：

> **1.报头和有效载荷如何分离？**
>
> **2.怎么分用？**

第二点很好说明：

> Tcp报头中也存在16位源端口号和16位目的端口号字段！！通过报头找到目的主机指定进程进行报文向上分用即可！

Tcp报头：

```
+--- --- --- --- -- -- -- ---- --- --- --- -- -- -- ---- --+--- --- -- -- -- ---- --- --- --- -- -- ---+
|                16位源端口号                |       16位目的端口号              |
+-----------------------------------------------------+-----------------------------------------+
|                                32位序号                                |
+---------------------------------------------------------------------------------------------- -+
|                                32位确认序号                                |
+-------------------+------+------+------+-----+----+------+---------------------------------+
|  4位首部长度 | U  |  A |  p |  R | S |  F |                                |
| -- -- -- -- -- -+|  R  | C | S | S | Y | I |       16位窗口大小       |
| 保留字段(6位) |  G  |  K  | H | T | N |  N |                                |
+-------------------+-------+------+------+-----+----+-----+---------------------------------+
|                        16位检验和                |    16位紧急指针          |
+-------------------+ --------------------- -------------   ---+ -- -- -- -- -- --- -- -- - -- - --+
|                                选项                                |
+---------------------- ---- --------------------  ----------------------  -------------------- ---+
|                                                                                                |
|                                数据                                |
|                                                                                                |
+-----------------------------------------------------------------------------------------------+
```

报头图：

<table>
<tr><td colspan="4">16位源端口号</td><td colspan="4">16位目的端口号</td></tr>
<tr><td colspan="8">32位序号</td></tr>
<tr><td colspan="8">32位确认序号</td></tr>
<tr><td>4位首部长度</td><td>保留6位</td><td>URG</td><td>ACK</td><td>PSH</td><td>RST</td><td>SYN</td><td>FIN</td><td>16位窗口大小</td></tr>
<tr><td colspan="4">16位检验</td><td colspan="4">16位紧急指针</td></tr>
<tr><td colspan="8">选线</td></tr>
<tr><td colspan="8">数据</td></tr>
</table>

报头属性介绍:

4位首部长度:

> 4比特位: max = 1111 ,min = 0000 ，即max = 15，min = 0;取值范围 [0,15]
>
> 基本单位：4字节
>
> 所以可以计算出Tcp报头长度范围：[0,60]（单位/字节）
>
> 而标准报头大小为：20字节，选项大小最大为:40字节

确认应答机制:

32位序号和确认序号

> 先提出：最新的一条信息永远不可能确定对方是否收到！只能通过手段确认历史信息对方是否收到！
>
> 先单独回答什么是可靠性：即作为发送方(我)，我不仅要确认历史信息对方是否收到，也要确认历史信息对方的接收状况，没收到的情况下：是丢包了还是根本没收到?
>
> 即：发送方发送一条信息，接收方需要发回应答！----->**确认应答机制**!
>
> **而确认应答机制是由双方OS自主实施的！！！**

> 但是，由于如果只是单纯的发一条应答一条这种模式效率太低！所以os一般会一次性发送**多条**信息，接收方发回多条应答！！
>
> 引出32位序号：
>
> 众所周知，网络的速度是会波动的，所以os一次性发送多条报文时，可能发送时是按照顺序发送的，但是如何确保接收方是按照顺序接收报文的呢？？是不是无法保证啊??! 所以Tcp报头引入了32位序号和确认序号字段来处理这种情况！！
>
> 即：**Tcp的按序到达策略**
>
> 发送方携带数据的序号，例如：第一次发送的数据序号为1000:表示本次数据范围为[1,1000] ,则接收方如果收到了，会在应答中填入32位确认序号，表示：这个确认序号之前的所有数据已经收到！！请发送方从确认序号 + 1的位置开始发送！
>
> 即确认序号为：1001。
>
> 由此根据序号字段，接收方就可以对接收到的报文进行**排序**！！
>
> **提出问题**：可是tcp报头为什么不只使用一个序号字段呢?何必加上确认序号字段？？
>
> 但是！别忘了，Tcp是**全双工**的啊！！！服务器是不是也可以给客户端发信息啊？如果服务器在确认应答的基础上也想给一个response呢？所以Tcp报头携带**序号和确认序号**两个字段！
>
> 所以上述所说的：**接受方在确认应答的基础上也想给一个response（数据）--->即：捎带应答**
>
> **综上所述：**发送方每次发送的请求中，Tcp报头都会携带对应的32位序号，用来表示当前发送的信息的唯一性，同样的，接收方每次做出应答时，应答的Tcp报头中的32位确认序号就是32位序号 + 1，**表示当前确认序号之前的数据已经全部收到！**
>
> ---

标记位：

作用：区分不同的Tcp报文类型！

> 1.ACK标记位
>
> 2.SYN标记位
>
> 3.FIN标记位
>
> 4.RST标记位

三次握手（初识）：

> 客户端发起建立连接的请求时会发送一个Tcp报头信息，其中的SYN标记位被置1，服务端收到后会回应客户端一个
>
> SYN应答 + ACK应答,客户端收到后再次向服务端发送一个带有将ACK标记位置为1的Tcp报头的信息。

> 客户端的connect接口本质上是发起三次握手，三次握手完成connect函数返回。服务端的accept函数在三次握手完成后返回本次完成的连接fd。
>
> ---

四次挥手（初识）：

> 客户端发起**请求断开连接**的请求，即本次tcp报头中的FIN标记位被置为1；即：**第一次挥手**
>
> 服务器收到FIN标记位，回应一个ACK应答 ,即：**第二次挥手**；
>
> 但是服务器也需要向客户端发起一个**请求断开连接**的请求，tcp报头的FIN标记位置为1；**即第三次挥手**！
>
> 客户端收到FIN请求，应答一个ACK，即第四次挥手！

超时重传机制：

> 如果作为接收方，如果没有收到上一条信息的ACK应答时，就认为对方没有收到报文！！
>
> 所以，发送方会等一段时间等待ACK应答！！！如果超过这个时间发送方还是没有收到应答，就认为对方没有收到报文！
>
> 如果判定超时！会触发超时重传机制！！
>
> **先提出一个问题**：有没有一种可能，如果作为发送方没有收到ACK的应答的原因不是丢包了，而是阻塞在网络中呢？？当触发超时重传后，发送方又重新发送了一遍历史报文，但是此时网络又畅通了，导致上一次(历史)的报文和重传的报文一起被接收方接收了怎么办？？
>
> 别忘了！tcp报头中是有32位序号的啊！！这个序号具有天然的去重的功能，如果判断数据**序号重复**就会直接丢弃！！
>
> **超时时间**一般是**动态**的！随着网络的状况动态变化！例如Linux下：初始500ms一次，如果再次触发则翻一倍：1000ms.....

连接管理机制：

三次握手（再识）：

> 理性理解三次握手：
>
> 三次握手：不能保证双方一定能建立好连接，而是经历三次握手，cs都认为连接已经建立好了！
>
> 因为，永远无法保证最新的一次SYN请求对方收到了！！

问题：如果三次握手的过程中出现异常了怎么办？，即服务端没收到客户端的ACK信息？双方如何得知？

> **引入RST标记位**：客户端自身建立好Tcp连接时，如果继续发送消息到服务端，但是服务端并没有收到客户端最后一次的ACK响应，此时，服务端会向客户端应答一次，这次应答的Tcp报头的标记位中的RST标记位和ACK被置为1，客户端收到服务端的这次应答后，就知道之前的Tcp连接建立失败了，此时客户端会释放异常的连接，重新发起三次握手建立连接
>
> ---

问题：Tcp为什么是三次握手？

> 要回答三次握手，首先要说明为什么不能是一次握手或者二次握手！
>
> 对于一次握手，二次握手，由于连接比较简单，Clinet可以伪造大量的SYN连接请求，向服务器发起连接请求，一次握手的话，server极有可能会在短时间内被打满相当多的Tcp链接，因为在Server看来，如果只有一次握手，当收到Client的SYN请求时，就默认双方都建立好了Tcp链接
>
> 一次握手容易让服务器出现严重的问题，甚至挂掉，二次握手同理，Client可以忽略来自Server的应答，继续SYN洪水，还是容易造成上述一次握手的问题。，但是对于三次握手，服务器虽然无法保证能防御SYN洪水，但是比一次，二次握手好一点，至少能保证Clinet一定在自己建立连接成功之前先建立连接成功的状态！
>
> ---
>
> **正面回答三次握手：**
>
> 1.双方验证全双工：验证网络的连通性，对于客户端来说，只使用最少的次数验证了双方的连通性，因为能收到服务器的ACK应答；对于服务器来说，也只用了**最少的次数**验证了双方的连通性，因为同理能收到来自客户端的ACK应答
>
> 2.建立双方TCP通信的共识意愿，客户端先发出SYN连接请求，服务器收到了，返回一个ACK应答，至此，客户端建立连接结束，但是Tcp连接是全双工的，服务器也要向客户端发送连接请求SYN，得到客户端的ACK应答后，即完成三次握手，而在其中服务器向客户端发送第一次ACK时会捎带SYN，即捎带应答；

四次挥手（再识）

> 服务端状态变化：CLOSE_WAIT--->LAST_ACK-->CLOSED
>
> 客户端状态变化：FIN_WAIT1---->FIN_WAIT2--->TIME_OUT--->CLOSED
>
> **为什么要断开连接需要四次挥手：**
>
> 跟三次握手思想 一致！

四次挥手状态变化：

> **重点的状态**：CLOSED、TIME_OUT
>
> **网络指令**：netstat -natp--查看服务器上所有tcp连接状态
>
> 注意！TCP协议规定：主动断开连接的一方要进入**TIME_OUT**状态
>
> **问题：处于TIME_OUT状态的时长是多大呢**？

> 处于TIME_OUT时长 == 网络中报文的存活最大时长 == 2MLS--->1MLS约为60s，不同系统可能不一样;
>
> **TIME_OUT状态的作用**：
>
> 1.保证历史的报文尽量消散！
>
> 2.保证对方能正常将连接关闭！保证最后一次ACK成功发出！

流量控制机制：

> **场景：**如果发送方发送数据的速度太快，而接收方应用层的业务处理速度较慢（读取缓冲区速度慢）的话，接收方的接收缓冲区会迅速被打满，此时，如果又来了新的报文会被丢弃！
>
> **提出疑问：**可是TCP不是具有超时重传机制吗？被丢弃的报文不是会超时重传吗？是的，但是不合理！！
>
> 为什么不合理？发送方发送的报文数据已经经过网络传输(网络状态良好)，消耗了网络资源的情况下经常因为对方缓冲区满了的情况下被丢弃，哪怕一直重传不仅消耗资源也毫无意义！
>
> **所以TCP引入了流量控制机制：**
>
> 主机A可以向主机B通告自己的接收能力！当然主机B也可以向主机A通告自己的接收能力！
>
> 这个接受能力是动态变化的！
>
> **如何通告对方？Tcp报头中的16位窗口大小字段！！意义是存放自身剩余接收缓冲区的剩余空间大小.**
>
> **如何填**？在每次的ACK应答中既将ACK标记位置为1，也填写自己的应答报头中的16位窗口大小字段！！！
>
> **问题1**：如果主机B的缓冲区满了，主机A难道会一直等待吗？主机A怎么实时的得知主机B的缓冲区有没有变化？反过来主机B对主机A呢？
>
> 主机A会定期发送窗口探测报文！主机B必定会应答一个ACK(确认应答机制)，ACK报文中是不是可以填写窗口字段啊！！直接更新即可。
>
> **问题2：**如果ACK应答中更新的窗口大小一直都是零怎么办？即接收方的接收缓冲区一直是满的，接收方上层一直没有取走数据呢？
>
> 引入**PSH**标记位，全名：PUSH,即要求对方主机**尽快**取走接收缓冲区的数据，当接收方主机通告的窗口大小一直为0的时候(PSH不一定只在这种情况下会置1，正常通信过程也可能存在！)，发送方会将报头的PSH标记位置为1，要求对方主机上层**尽快**取走接收缓冲区的数据。
>
> ---

16位紧急指针：

> **URG标记位：**

> 如果tcp报头中设置了URG标记位，意味着16位紧急指针字段有效，接收方需提取该字段，16位紧急指针指向正文数据的某个位置！大小为1字节，一般为状态码信息，接收方拿到16位紧急指针后就会有效读取正文数据中的指针指向的位置！
>
> 如何在应用层设置紧急指针？send!
>
> ```
> MSG_OOB
>  Sends  out-of-band data on sockets that support this notion (e.g., of type SOCK_STREAM); the underlying protocol must also
>  support out-of-band data.
> ```
>
> 参数flags设置该标记位即可设置带外数据
>
> **recv函数同理**
>
> ---

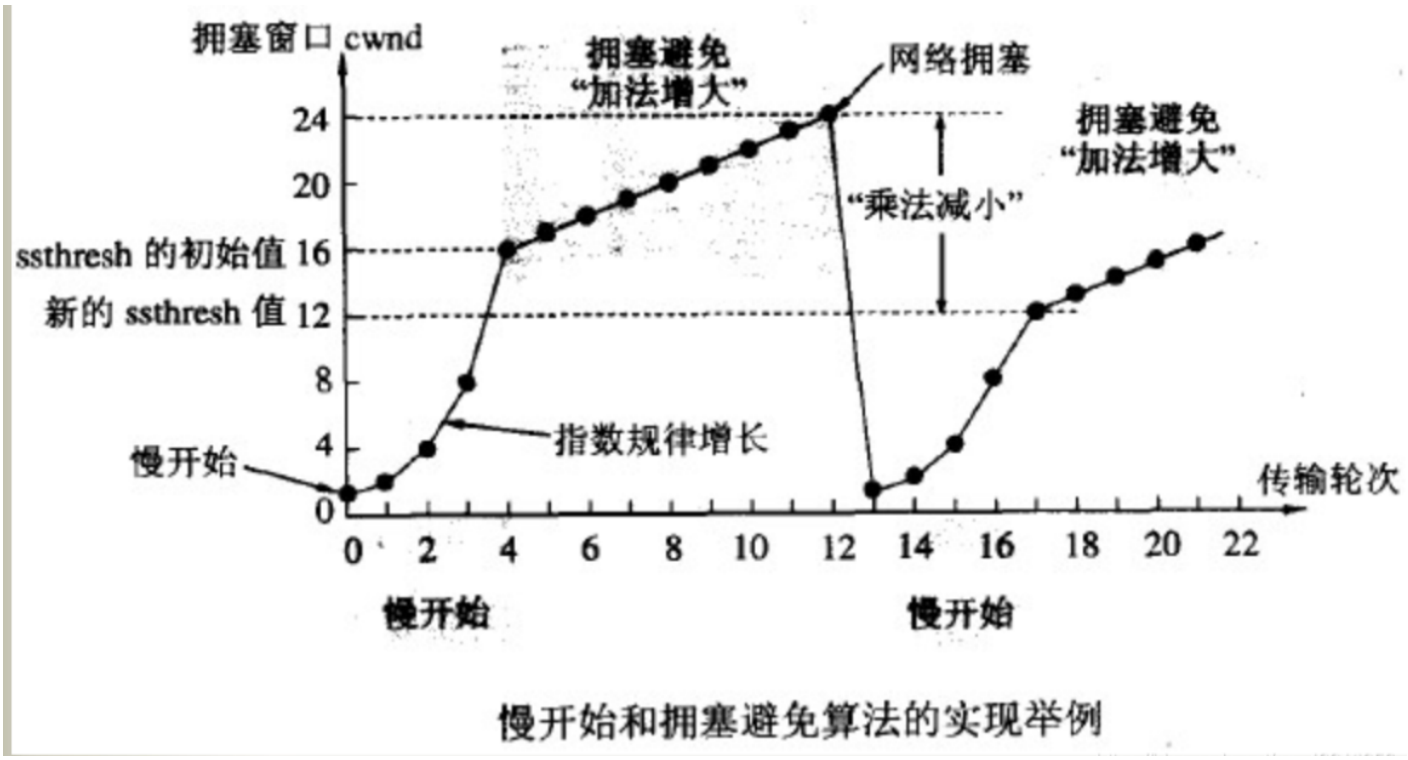
滑动窗口：

> 实际上的发送方和接收方互发数据不是简单的发送一条syn，应答一个ACK，而是发送方可以连续发送多条报文而不用收到ACK应答！
>
> 那么这里的**多条**是什么意思?有没有一个标准？---滑动窗口来衡量！！
>
> ---
>
> **滑动窗口的本质是发送缓冲区的一部分！凡是在滑动窗内的数据可以直接发送，不需要立即收到应答！**
>
> 在linux内核中，发送缓冲区存在2个指针，分别指向窗口开始和窗口末尾。
>
> 即在发送缓冲区中，一定存在3个部分！如果窗口开始为start，结尾为end.
>
> **第一个部分**为已发送的数据部分，范围[0,start)
>
> **第二部分**为滑动窗口覆盖的数据部分，即可发送的数据部分，范围[start,end]
>
> **第三部分**为待发送的数据部分，范围[end+1,缓冲区结尾]
>
> ---
>
> **问题：**
>
> **1.对于超时重传，既然tcp会对发送的，没有收到应答的报文进行超时重传，那么必定会保存历史发送的数据，如何保存？**
>
> 在滑动窗口中等待确认！如果没收到应答，滑动窗口不会向右移动！
>
> **2.滑动窗口只能向右滑动吗？可以变小吗？可以变大吗？可以为0吗？**
>
> 滑动窗口只能向右滑动，无法左移；
>
> 可以变大，自然也可以变小，当然也可以为0，换言之，**滑动窗口是可以动态变化的！**
>
> ---
>
> **理解滑动窗口：**

> 发送缓冲区中存在2个指针，一个start_ptr指向开始，end_ptr指向结尾，通过2个指针的**右移速度不同**来控制滑动窗口的大小！
>
> **丢包问题：假设为发送[1000,5000]的报文**
>
> **1.最左侧报文丢包呢？[1000,2000]范围的报文**
>
> 别忘记了，Tcp报头中存在32位确认序号字段啊！这个确认序号的意义是：确认序号之前的报文**全部**收到了！
>
> 如果最左侧报文丢失，作为接收方，哪怕收到了后面的报文，它能将确认序号设置为5001吗？？很明显不能的啊!!!所以，如果最左侧报文丢失了的话，接受方只能将确认序号填为1001！
>
> 此时滑动窗口不进行移动，会重发[1000,2000]的报文数据，即此时如果连续收到**3次**左侧报文丢失的ACK应答时，发送方会**立即**补发丢包的数据[1000,2000]，即触发**快重传机制**！！而不是超时重传，因为超时重传太慢了！！
>
> **疑问：一定要3次才会触发快重传吗？**
>
> 是的，如果没有达到3次，只会触发超时重传机制，即快重传机制是用来提高传输效率的！！超时重传是用来兜底的！！！
>
> **2.中间报文丢失呢？**
>
> 本质还是最左侧报文丢失问题！同样的解决方案！！
>
> **3.最右侧报文丢失呢？**
>
> 本质还是最左侧报文丢失问题！同样的解决方案！！
>
> **补充**：可以简单将缓冲区理解成环形的结构，所以不会溢出~
>
> ---

拥塞控制机制（慢启动算法)：

> 前面所说所有机制都只是双方主机的传输数据的策略，并没有涉及到网络状态本身！
>
> 如果双方主机通信时，网络状态不好，刚开始就发送大量的数据的话，必然会导致网络状态更加糟糕！所以tcp引入了**慢启动机制！**
>
> **拥塞窗口：**当发送的数据不超过这个窗口大小时，网络不会发生拥塞。
>
> **所以完善一下滑动窗口的概念，即滑动窗口 = min(接收方的窗口接受能力 + 拥塞窗口大小);**
>
> **解释：**如果网络状态非常好，此时滑动窗口大小就是接收方的窗口接受能力大小，如果网络状态很差，那么滑动窗口大小就是拥塞窗口的大小！
>
> **慢启动算法：**



延迟应答机制：

> 主机A向主机B发送报文，主机B收到后不会立即应答ACK，而是等待一段时间再发送！(时间 < 超时时间)
>
> 为什么要等？联系滑动窗口方面！！！在等待的这段时间中，接收方的上层大概率会取走更多的数据，所以不就可以通告更大的窗口给对方吗？？？

粘包问题：

> 使用应用层协议分割报文即可！！！

Tcp异常问题

> **进程终止:** 进程终止会释放文件描述符, 仍然可以发送 FIN. 和正常关闭没有什么区别.
>
> **机器重启:** 和进程终止的情况相同.
>
> **机器掉电/网线断开:** 接收端认为连接还在, 一旦接收端有写入操作, 接收端发现连接已
>
> 经不在了, 就会进行 reset. 即使没有写入操作, **TCP 自己也内置了一个保活定时器, 会**
>
> **定期询问对方是否还在. 如果对方不在, 也会把连接释放.**
>
> 另外, 应用层的某些协议, 也有一些这样的检测机制. 例如 HTTP 长连接中,
>
> 也会定期检 测对方的状态. 例如 QQ, 在 QQ 断线之后, 也会定期尝试重新连接.

-- ---- ---- ----------done----------